

TL

TECHNOLOGIS S.A.

TechnoLogis S.A.

*Système de Gestion Intelligente d'Inventaire et d'Actifs*

## Document de Spécification du Projet

---

Mai 2026

| INFORMATIONS DU PROJET                                                        |
|-------------------------------------------------------------------------------|
| <b>Cours :</b> Framework .NET de Microsoft                                    |
| <b>Enseignant :</b> Jean Gregory SOLIDE                                       |
| <b>Préparé par :</b> Dawens Holtzman PIERRE                                   |
| <b>Programme :</b> MBDS-Haiti 2025-2026                                       |
| <b>Client :</b> Mme Valdaya PRINCIVIL — Directrice Générale, TechnoLogis S.A. |



## Table des matières

---

### Table des matières

|                                                            |    |
|------------------------------------------------------------|----|
| PDR 6 — Gestion Intelligente d’Inventaire & d’Actifs ..... | iv |
| 1. Aperçu .....                                            | iv |
| 2. Problématique .....                                     | iv |
| 3. Objectifs .....                                         | iv |
| 4. Fonctionnalités Clés .....                              | iv |
| 5. Utilisateurs Cibles.....                                | iv |
| 6. Contraintes / Hypothèses .....                          | iv |
| 1. INTRODUCTION.....                                       | 5  |
| 1.1 Contexte et problématique.....                         | 5  |
| 1.2 Présentation du projet.....                            | 5  |
| 1.3 Objectifs .....                                        | 6  |
| 1.3.1 Objectif général.....                                | 6  |
| 1.3.2 Objectifs spécifiques .....                          | 6  |
| 2. CADRAGE DU PROJET .....                                 | 7  |
| 2.1 Cadre conceptuel métier .....                          | 7  |
| 2.2 Utilisateurs cibles .....                              | 8  |
| 3. MÉTHODOLOGIE .....                                      | 9  |
| 3.1 Analyse de l'existant .....                            | 9  |
| 3.2 Analyse des besoins .....                              | 9  |
| 3.2.1 Besoins fonctionnels .....                           | 9  |
| 3.2.2 Besoins non fonctionnels.....                        | 9  |
| 3.3 Choix de la solution technique.....                    | 10 |
| 3.4 Approche de conception des données .....               | 10 |
| 4. SPÉCIFICATIONS FONCTIONNELLES .....                     | 11 |
| 4.1 Modules du système .....                               | 11 |
| Module 1 — Catégories .....                                | 11 |
| Module 2 — Produits.....                                   | 11 |
| Module 3 — Actifs.....                                     | 11 |
| Module 4 — Stock.....                                      | 11 |

|                                               |    |
|-----------------------------------------------|----|
| Module 5 — Services / Départements.....       | 12 |
| Module 6 — Employés.....                      | 12 |
| Module 7 — Affectations.....                  | 12 |
| Module 8 — Maintenances.....                  | 12 |
| Module 9 — Rapports.....                      | 12 |
| Module 10 — Audit.....                        | 12 |
| Module 11 — Utilisateurs.....                 | 13 |
| Module 12 — Dashboard.....                    | 13 |
| 4.2 Cas d'utilisation.....                    | 13 |
| 4.3 Règles de gestion.....                    | 14 |
| 5. MODÉLISATION DES DONNÉES.....              | 15 |
| 5.1 Identification des entités.....           | 15 |
| 5.2 Description des entités et attributs..... | 15 |
| Categorie.....                                | 15 |
| Produit.....                                  | 15 |
| Actif.....                                    | 16 |
| Stock.....                                    | 16 |
| MouvementStock.....                           | 16 |
| Service.....                                  | 16 |
| Employe.....                                  | 16 |
| Affectation.....                              | 17 |
| Maintenance.....                              | 17 |
| Utilisateur (ASP.NET Identity).....           | 17 |
| AuditLog.....                                 | 17 |
| 5.3 Modèle Conceptuel de Données (MCD).....   | 18 |
| .....                                         | 19 |
| Modèle Logique de Données (MLD).....          | 1  |
| 5.5 Modèle Physique de Données (MPD).....     | 1  |
| 6. ARCHITECTURE TECHNIQUE.....                | 2  |
| 6.1 Architecture globale.....                 | 2  |
| 6.2 Organisation en couches.....              | 2  |
| Couche Domain (GestionInventaire.Domain)..... | 2  |
| Couche DAL (GestionInventaire.DAL).....       | 2  |
| Couche BLL (GestionInventaire.BLL).....       | 2  |
| Couche Web (GestionInventaire.Web).....       | 3  |

|                                              |   |
|----------------------------------------------|---|
| 6.3 Flux de traitement.....                  | 3 |
| 6.4 Technologies et outils .....             | 3 |
| 7. CONTRAINTES ET HYPOTHÈSES.....            | 4 |
| 7.1 Contraintes techniques.....              | 4 |
| 7.2 Contraintes de performance.....          | 4 |
| 7.3 Contraintes de sécurité.....             | 4 |
| 7.4 Hypothèses .....                         | 4 |
| 8. NORMALISATION DE LA BASE DE DONNÉES ..... | 5 |
| 8.1 Première forme normale (1FN).....        | 5 |
| 8.2 Deuxième forme normale (2FN) .....       | 5 |
| 8.3 Troisième forme normale (3FN).....       | 5 |

## **PDR 6 — Gestion Intelligente d'Inventaire & d'Actifs**

### **1. Aperçu**

Système de suivi des équipements, des niveaux de stock, des affectations et des cycles de maintenance.

### **2. Problématique**

Perte d'équipements, incohérences d'inventaire et maintenance non planifiée entraînent des coûts et des risques opérationnels.

### **3. Objectifs**

- Visibilité temps réel des actifs
- Traçabilité du cycle de vie et des affectations
- Automatisation des rappels de maintenance

### **4. Fonctionnalités Clés**

- Inventaire par catégories et statuts
- Scan QR/barcode pour identification rapide
- Workflow d'emprunt/retour
- Planification de maintenance
- Alertes de seuil de stock
- Historique d'affectation et d'audit

### **5. Utilisateurs Cibles**

- Services IT
- Laboratoires
- Gestion des installations
- PME

### **6. Contraintes / Hypothèses**

- Journaux d'audit précis et sécurisés
- Dépendance aux performances caméra pour le scan
- Métadonnées spécifiques pour certains actifs

# 1. INTRODUCTION

## 1.1 Contexte et problématique

Dans un contexte de transformation numérique croissante, Madame Valdaya PRINCIVIL, Directrice Générale de l'entreprise TechnoLogis S.A., est confrontée à des difficultés majeures liées à la gestion de ses équipements et de ses stocks.

Actuellement, l'organisation repose sur des outils manuels et des systèmes hétérogènes (fichiers Excel, registres papier), ce qui entraîne un manque de visibilité globale sur les actifs, des incohérences dans les données d'inventaire et une difficulté à assurer un suivi fiable des affectations et des opérations de maintenance.

Cette situation engendre plusieurs conséquences critiques, notamment des pertes d'équipements, des achats redondants, des interruptions d'activité dues à des pannes non anticipées, ainsi qu'un manque de traçabilité dans l'utilisation des ressources.

De plus, l'absence d'un système centralisé empêche une gestion efficace du cycle de vie des actifs, limitant ainsi la capacité de l'entreprise à prendre des décisions éclairées basées sur des données fiables.

Face à ces enjeux, Madame PRINCIVIL a lancé un appel d'offres visant à concevoir et mettre en œuvre une solution informatique capable de centraliser, sécuriser et optimiser la gestion des actifs et des stocks.

## 1.2 Présentation du projet

Le projet proposé, intitulé « Système de Gestion Intelligente d'Inventaire et d'Actifs », consiste en la conception et le développement d'une application web permettant de gérer de manière centralisée les produits, les équipements (actifs), les stocks, les affectations et les opérations de maintenance.

Cette solution vise à offrir une plateforme unifiée permettant de suivre le cycle de vie complet des actifs, depuis leur acquisition (via le module Produits) jusqu'à leur mise hors service, tout en assurant une traçabilité complète des opérations.

Le système intègre des fonctionnalités avancées telles que la génération automatique de codes inventaire, la gestion des mouvements de stock, l'affectation des actifs aux employés par département, la planification des maintenances et la génération de rapports opérationnels avec export CSV.

## 1.3 Objectifs

### 1.3.1 Objectif général

Mettre en place un système intelligent, centralisé et performant de gestion d'inventaire et des actifs, permettant d'améliorer la visibilité, la traçabilité et l'efficacité opérationnelle au sein de l'organisation.

### 1.3.2 Objectifs spécifiques

- Assurer une visibilité en temps réel sur l'ensemble des actifs et des niveaux de stock
- Garantir une traçabilité complète du cycle de vie des équipements (acquisition, affectation, maintenance, réforme)
- Automatiser la génération des codes inventaire et la création des stocks lors de l'enregistrement des produits
- Automatiser la planification et le suivi des opérations de maintenance préventive et corrective
- Mettre en place un système d'alertes pour les seuils de stock critiques et les maintenances à venir
- Optimiser la gestion des affectations des actifs par employé et par service/département
- Réduire les erreurs humaines et les incohérences dans les données d'inventaire
- Fournir des données fiables et historisées pour faciliter la prise de décision via des rapports et l'audit
- Concevoir une solution évolutive, adaptable aux besoins de différents types d'organisations



## 2. CADRAGE DU PROJET

### 2.1 Cadre conceptuel métier

Dans le cadre de la conception du système, il est essentiel de définir un cadre conceptuel métier permettant d'harmoniser la compréhension des différents acteurs impliqués dans le projet.

Les concepts métier clés retenus sont les suivants :

- Un produit représente un modèle ou type d'équipement du catalogue de l'entreprise (ex. Laptop Dell XPS 15, Switch HP 24 ports). Il appartient à une catégorie et dispose d'un stock associé. La création d'un produit déclenche automatiquement la génération des actifs physiques correspondants. Produit :
- Un actif désigne une unité physique d'un produit, identifiée par un code inventaire unique généré automatiquement (format : 1ère lettre du produit + numéro séquentiel, ex. L000001). Chaque actif possède un statut (Disponible, Affecté, En maintenance, Hors service) et une localisation. Actif :
- Une catégorie permet de classer les produits selon leur nature ou leur fonction (informatique, réseau, mobilier, etc.). Catégorie :
- Le stock représente la quantité disponible d'un produit donné. Il est créé automatiquement lors de l'enregistrement d'un produit et mis à jour lors des approvisionnements et des affectations. Stock :
- Un employé est une personne physique appartenant à un service/département de l'entreprise. C'est à lui que sont affectés les actifs, et non à l'utilisateur du système. Employé :
- Un service (ou département) regroupe les employés selon leur appartenance organisationnelle (Informatique, RH, Finance, Direction, etc.). Il permet de structurer les affectations. Service :
- L'affectation correspond à l'attribution d'un actif à un employé. Elle comporte une date de début et une date de fin optionnelle. Lors d'une affectation, le stock du produit correspondant diminue d'une unité. Lors du retour, il est réincrémenté. Affectation :
- Un utilisateur désigne toute personne autorisée à interagir avec le système (Admin, Gestionnaire, Technicien). La gestion des utilisateurs est assurée via ASP.NET Identity avec envoi d'invitation par email. Utilisateur :
- La maintenance regroupe les actions visant à assurer le bon fonctionnement des actifs. Elle peut être planifiée (statut : Planifiée), en cours ou terminée. Elle est associée à un coût estimé. Maintenance :

- Un mouvement correspond à toute opération d'entrée ou de sortie de stock. Les mouvements sont générés automatiquement lors des approvisionnements, des affectations et des retours. Mouvement de stock :
- Le journal d'audit trace toutes les actions significatives effectuées dans le système (créations, modifications, suppressions, affectations, maintenances) avec l'utilisateur responsable et l'horodatage. Audit :
- Le module de reporting agrège les données de tous les modules pour fournir un tableau de bord opérationnel exportable en CSV. Rapport :

## 2.2 Utilisateurs cibles

Le système est destiné à différents profils d'utilisateurs avec des niveaux d'accès distincts :

| Rôle                | Accès         | Responsabilités                                                                      |
|---------------------|---------------|--------------------------------------------------------------------------------------|
| <b>Admin</b>        | Accès complet | Configuration système, gestion des utilisateurs, audit, rapports, tous les modules   |
| <b>Gestionnaire</b> | Accès étendu  | Gestion des produits, actifs, stocks, affectations, employés, maintenances, rapports |
| <b>Technicien</b>   | Accès limité  | Consultation des actifs, gestion des maintenances, affectations                      |

## 3. MÉTHODOLOGIE

### 3.1 Analyse de l'existant

La première étape a consisté à analyser le système actuel de gestion des actifs et des stocks au sein de l'entreprise. Il en ressort que la gestion repose principalement sur des fichiers Excel et des supports manuels, sans centralisation des données. Cette situation entraîne plusieurs insuffisances :

- Absence de visibilité globale sur les actifs et les niveaux de stock
- Difficulté à suivre les affectations des équipements par employé et par département
- Manque de traçabilité des opérations (mouvements, maintenance, historique)
- Risque élevé d'erreurs humaines et d'incohérences dans les données
- Absence de mécanisme d'alerte pour les maintenances ou les seuils critiques
- Absence de ressources spécialisées en administration de bases de données

### 3.2 Analyse des besoins

#### 3.2.1 Besoins fonctionnels

- Gestion du catalogue produits avec création automatique du stock et des actifs
- Gestion des actifs avec codes inventaire générés automatiquement (format D6)
- Approvisionnement en masse : génération de N actifs depuis la fiche produit
- Suivi des niveaux de stock et des mouvements (entrées/sorties automatiques)
- Gestion des services/départements (CRUD)
- Gestion des employés rattachés à un service avec liste déroulante dynamique
- Gestion des affectations avec impact automatique sur le stock
- Planification et suivi des opérations de maintenance avec gestion des coûts
- Changement de statut des maintenances (Planifiée → En cours → Terminée)
- Journal d'audit traçant toutes les actions par utilisateur connecté
- Rapports opérationnels consolidés (inventaire, stock, affectations, coûts maintenance) avec export CSV
- Gestion des utilisateurs via ASP.NET Identity (invitation par email, rôles, verrouillage)

#### 3.2.2 Besoins non fonctionnels

- Performance : temps de réponse rapides, même pour de grands volumes d'actifs (jusqu'à 999 999 par produit)

- Sécurité : authentification via ASP.NET Identity, gestion des rôles, protection des journaux d'audit
- Fiabilité : transactions atomiques (un seul SaveAsync pour les opérations groupées)
- Évolutivité : architecture en couches permettant l'ajout de nouvelles fonctionnalités
- Ergonomie : interface thème sombre, navigation par menu principal + dropdown « Plus »

### 3.3 Choix de la solution technique

L'application est développée avec la plateforme ASP.NET Core MVC en architecture 4 couches (Domain, DAL, BLL, Web), Entity Framework Core en approche Code First, AutoMapper pour les mappings DTO/ViewModel, et Microsoft SQL Server comme SGBD.

ASP.NET Identity gère l'authentification, les rôles (Admin, Gestionnaire, Technicien) et le cycle de vie des comptes utilisateurs avec envoi d'invitations par email.

### 3.4 Approche de conception des données

Une approche Code First a été adoptée. Le modèle de données est défini directement dans les classes du domaine métier (GestionInventaire.Domain), puis la structure de la base de données est générée automatiquement via les migrations Entity Framework Core. Cette approche garantit la cohérence entre la logique métier et la base de données, et facilite l'évolution du schéma.

## 4. SPÉCIFICATIONS FONCTIONNELLES

### 4.1 Modules du système

Le système est structuré autour des modules suivants :

#### Module 1 — Catégories

Gestion CRUD des catégories de produits. Chaque catégorie regroupe des produits selon leur nature. La suppression n'est autorisée que si aucun produit n'y est rattaché. Accessible depuis le menu Produits via un lien « Nouvelle catégorie » (nouvel onglet).

#### Module 2 — Produits

Gestion du catalogue produits. La création d'un produit déclenche automatiquement en une seule transaction : (1) création du Produit, (2) création du Stock avec seuil d'alerte, (3) génération en masse des actifs physiques avec codes inventaire séquentiels, (4) création du mouvement de stock d'entrée.

Le bouton « Approvisionner » permet d'ajouter des actifs supplémentaires à un produit existant. La suppression d'un produit est bloquée si des actifs encore en service lui sont rattachés.

#### Module 3 — Actifs

Les actifs sont générés automatiquement depuis le module Produits. Chaque actif possède un code inventaire unique (format : 1ère lettre du produit + 6 chiffres, ex. L000001). Seules la localisation et le statut (Disponible/Hors service) sont modifiables manuellement. Les statuts Affecté et En maintenance sont gérés automatiquement par les modules Affectations et Maintenances.

La page Index affiche les actifs avec filtrage par statut (KPI cliquables) : Disponible, Affecté, En maintenance, Hors service.

#### Module 4 — Stock

Suivi des quantités disponibles par produit. Le stock est créé automatiquement avec chaque produit. Les mouvements (entrées/sorties) sont générés automatiquement lors des approvisionnements, des affectations et des retours. L'historique complet des mouvements est accessible depuis la liste des stocks via le bouton « Historique ».

## Module 5 — Services / Départements

Gestion CRUD des services et départements de l'entreprise. Chaque service dispose d'un nom et d'une description optionnelle. La suppression est bloquée si des employés y sont rattachés. Ce module alimente la liste déroulante dans le formulaire de création/modification des employés.

## Module 6 — Employés

Gestion CRUD du personnel. Chaque employé est rattaché à un service via une liste déroulante dynamique (chargée depuis le module Services). Les champs disponibles sont : Nom, Prénom, Email, Téléphone, Service. La suppression est bloquée si l'employé a des affectations actives.

## Module 7 — Affectations

Gestion des attributions d'actifs aux employés. Seuls les actifs avec le statut « Disponible » peuvent être affectés. Lors d'une affectation : l'actif passe en statut « Affecté » et le stock du produit diminue d'une unité. Lors du retour : l'actif repasse en « Disponible » et le stock est réincrémenté.

## Module 8 — Maintenances

Planification et suivi des interventions sur les actifs. Une maintenance peut être créée pour tout actif non hors service. Lors de la planification, l'actif passe en statut « En maintenance ». Le cycle de statut est : Planifiée → En cours → Terminée. Lors de la clôture (Terminée), l'actif repasse automatiquement en « Disponible ». Chaque maintenance enregistre un coût estimé.

## Module 9 — Rapports

Tableau de bord opérationnel consolidé en 4 sections accordéon : (1) Inventaire des actifs avec filtrage par statut, (2) État du stock avec identification des stocks critiques et épuisés, (3) Affectations actives avec durée par employé, (4) Maintenances et coûts totaux. Chaque section est exportable en CSV avec encodage UTF-8.

## Module 10 — Audit

Journal d'audit traçant toutes les actions significatives avec l'utilisateur connecté (via IHttpContextAccessor + Claims), la date/heure, l'entité concernée et son identifiant. Filtrable par mot-clé, type d'action et plage de dates. Accès réservé aux Admins.

## Module 11 — Utilisateurs

Gestion des comptes via ASP.NET Identity. La création d'un compte génère un mot de passe temporaire aléatoire et envoie un email d'invitation avec un lien de réinitialisation de mot de passe. Les Admins peuvent modifier le rôle, verrouiller/déverrouiller les comptes. La suppression du dernier Admin est bloquée.

## Module 12 — Dashboard

Tableau de bord principal affichant les KPI en temps réel : total actifs par statut, stocks critiques, alertes maintenances urgentes ( $\leq 3$  jours), activités récentes depuis le journal d'audit.

### 4.2 Cas d'utilisation

| Use Case                   | Description                                                                       |
|----------------------------|-----------------------------------------------------------------------------------|
| Créer un produit           | Créer le produit + stock + actifs en masse en une seule transaction               |
| Approvisionner un produit  | Ajouter des actifs supplémentaires à un produit existant depuis la liste Produits |
| Consulter l'inventaire     | Afficher les actifs avec filtrage par statut via les KPI cliquables               |
| Modifier un actif          | Modifier la localisation et le statut (Disponible/Hors service uniquement)        |
| Gérer les services         | Créer, modifier, supprimer les départements de l'entreprise                       |
| Affecter un actif          | Attribuer un actif disponible à un employé avec mise à jour automatique du stock  |
| Retourner un actif         | Clôturer une affectation et remettre l'actif en Disponible                        |
| Planifier une maintenance  | Créer une intervention sur un actif — passe l'actif en En maintenance             |
| Changer statut maintenance | Planifiée → En cours → Terminée (libère l'actif à la clôture)                     |
| Consulter les rapports     | Afficher le tableau de bord 4 sections et exporter en CSV                         |
| Consulter l'audit          | Rechercher dans le journal d'audit avec filtres                                   |
| Créer un utilisateur       | Inviter un nouvel utilisateur par email avec lien de définition du mot de passe   |
| Verrouiller un utilisateur | Bloquer l'accès d'un compte sans le supprimer                                     |

### 4.3 Règles de gestion

- Un actif ne peut être affecté que s'il est en statut Disponible
- Un actif en maintenance ou affecté ne peut pas être supprimé
- Toute affectation doit être associée à un employé existant et comporter une date de début
- Un produit appartient à une seule catégorie ; un actif appartient à un seul produit
- Les mouvements de stock sont générés automatiquement et ne peuvent pas être modifiés manuellement
- Le stock ne peut jamais être négatif
- Une maintenance doit être associée à un actif non hors service
- Le statut Affecté et En maintenance ne sont pas modifiables manuellement dans le module Actifs
- La suppression d'un produit est bloquée si des actifs non hors service y sont rattachés
- La suppression d'un service est bloquée si des employés y sont rattachés
- La suppression du dernier administrateur est bloquée
- Toutes les actions critiques sont tracées dans le journal d'audit avec l'utilisateur connecté
- Les codes inventaire sont générés au format : [1ère lettre du produit][numéro D6], ex. L000001



## 5. MODÉLISATION DES DONNÉES

### 5.1 Identification des entités

À partir de l'analyse métier et des spécifications fonctionnelles, les entités suivantes ont été identifiées :

| Entité         | Description                                                                             |
|----------------|-----------------------------------------------------------------------------------------|
| Categorie      | Classifie les produits selon leur nature (informatique, réseau, mobilier, etc.)         |
| Produit        | Modèle ou type d'équipement du catalogue. Chaque produit génère un stock et des actifs. |
| Actif          | Unité physique d'un produit, avec code inventaire unique généré automatiquement.        |
| Stock          | Quantité disponible d'un produit. Créé automatiquement à la création du produit.        |
| MouvementStock | Entrée ou sortie de stock générée automatiquement par les opérations système.           |
| Service        | Département ou service de l'entreprise auquel appartient un employé.                    |
| Employe        | Membre du personnel rattaché à un service. Destinataire des affectations d'actifs.      |
| Affectation    | Attribution d'un actif à un employé pour une période donnée.                            |
| Maintenance    | Intervention planifiée ou corrective sur un actif, avec statut et coût.                 |
| Utilisateur    | Compte système géré par ASP.NET Identity avec rôle (Admin/Gestionnaire/Technicien).     |
| AuditLog       | Trace de toutes les actions significatives effectuées dans le système.                  |

### 5.2 Description des entités et attributs

#### Categorie

- IdCategorie (clé primaire, INT)
- NomCategorie (VARCHAR 100, obligatoire, unique)
- Description (VARCHAR 300, optionnel)

#### Produit

- IdProduit (clé primaire, INT)

- NomProduit (VARCHAR 100, obligatoire)
- Description (VARCHAR 500, optionnel)
- IdCategorie (clé étrangère → Categorie)

### Actif

- IdActif (clé primaire, INT)
- CodeInventaire (VARCHAR 20, unique, généré automatiquement au format [Lettre][D6])
- Statut (ENUM : Disponible | Affecte | EnMaintenance | HorsService)
- Localisation (VARCHAR 100, obligatoire, min. 3 caractères)
- DateAcquisition (DATE, valeur par défaut = date du jour)
- IdProduit (clé étrangère → Produit)

### Stock

- IdStock (clé primaire, INT)
- IdProduit (clé étrangère → Produit, unique)
- Quantite (INT,  $\geq 0$ )
- SeuilAlerte (INT,  $\geq 0$ , seuil en dessous duquel une alerte est générée)

### MouvementStock

- IdMouvement (clé primaire, INT)
- IdStock (clé étrangère → Stock)
- Type (ENUM : Entree | Sortie)
- Quantite (INT,  $> 0$ )
- DateMouvement (DATETIME)

### Service

- IdService (clé primaire, INT)
- NomService (VARCHAR 100, obligatoire, unique)
- Description (VARCHAR 300, optionnel)

### Employe

- IdEmploye (clé primaire, INT)
- Nom (VARCHAR 100, obligatoire)
- Prenom (VARCHAR 100, obligatoire)

- Email (VARCHAR 150, optionnel)
- Telephone (VARCHAR 20, optionnel)
- IdService (clé étrangère → Service, optionnel)

### Affectation

- IdAffectation (clé primaire, INT)
- IdActif (clé étrangère → Actif)
- IdEmploye (clé étrangère → Employe)
- DateDebut (DATETIME, obligatoire)
- DateFin (DATETIME, null si affectation active)
- EstActive (calculé : DateFin IS NULL)

### Maintenance

- IdMaintenance (clé primaire, INT)
- IdActif (clé étrangère → Actif)
- DateMaintenance (DATE, obligatoire)
- Statut (ENUM : Planifiee | EnCours | Terminee)
- Description (VARCHAR 500, obligatoire)
- Cout (DECIMAL,  $\geq 0$ , coût estimé en HTG)

### Utilisateur (ASP.NET Identity)

- Id (clé primaire, string GUID)
- Nom (VARCHAR 100)
- Prenom (VARCHAR 100)
- Email (VARCHAR 256, unique)
- Telephone (VARCHAR 20, optionnel)
- EmailConfirmed (BOOLEAN)
- LockoutEnd (DATETIME, null si compte actif)
- Rôle : Admin | Gestionnaire | Technicien (table AspNetRoles)

### AuditLog

- IdAuditLog (clé primaire, INT)
- Action (VARCHAR 200, description de l'action effectuée)

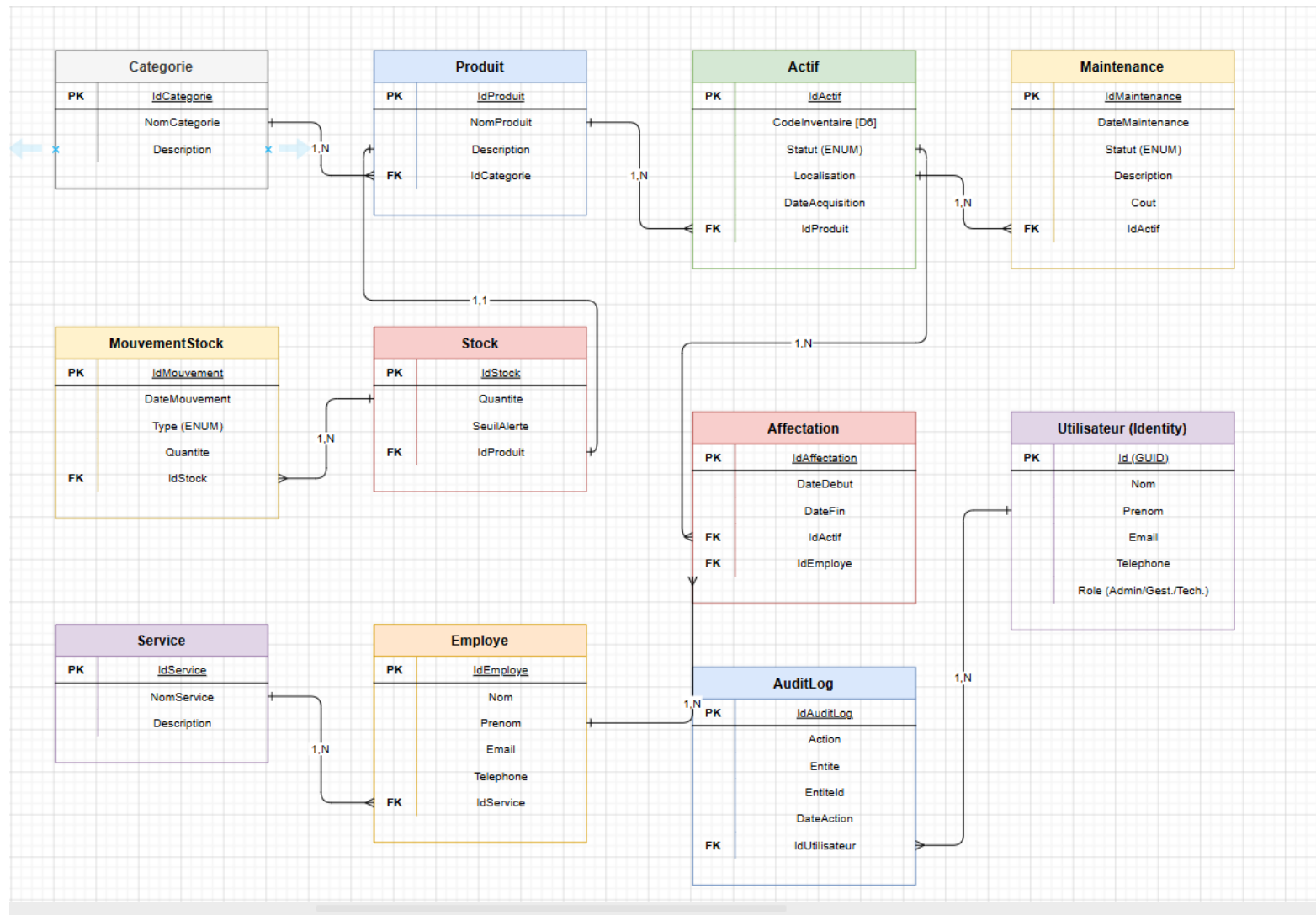
- Entite (VARCHAR 100, nom de l'entité concernée)
- EntiteId (INT, identifiant de l'entité concernée)
- DateAction (DATETIME)
- IdUtilisateur (clé étrangère → Utilisateur, string GUID)

### 5.3 Modèle Conceptuel de Données (MCD)

Les principales relations entre entités sont les suivantes :

- Catégorie — Produit : une catégorie contient plusieurs produits (1,N) ; un produit appartient à une seule catégorie (1,1).
- Produit — Actif : un produit génère plusieurs actifs (1,N) ; un actif correspond à un seul produit (1,1).
- Produit — Stock : un produit possède un seul enregistrement de stock (1,1).
- Stock — MouvementStock : un stock a plusieurs mouvements (1,N).
- Service — Employe : un service regroupe plusieurs employés (1,N) ; un employé appartient à un seul service (0,1).
- Employe — Affectation : un employé peut recevoir plusieurs actifs (1,N).
- Actif — Affectation : un actif peut faire l'objet de plusieurs affectations successives (1,N).
- Actif — Maintenance : un actif peut avoir plusieurs interventions de maintenance (1,N).
- Utilisateur — AuditLog : un utilisateur génère plusieurs entrées d'audit (1,N).

#### Représentation schématique du MCD



## Modèle Logique de Données (MLD)

Chaque entité devient une table ; les relations sont matérialisées par des clés étrangères :

- CATEGORIE (IdCategorie, NomCategorie, Description)
- PRODUIT (IdProduit, NomProduit, Description, #IdCategorie)
- ACTIF (IdActif, CodeInventaire, Statut, Localisation, DateAcquisition, #IdProduit)
- STOCK (IdStock, Quantite, SeuilAlerte, #IdProduit)
- MOUVEMENT\_STOCK (IdMouvement, Type, Quantite, DateMouvement, #IdStock)
- SERVICE (IdService, NomService, Description)
- EMPLOYE (IdEmploye, Nom, Prenom, Email, Telephone, #IdService)
- AFFECTATION (IdAffectation, DateDebut, DateFin, #IdActif, #IdEmploye)
- MAINTENANCE (IdMaintenance, DateMaintenance, Statut, Description, Cout, #IdActif)
- UTILISATEUR (Id, Nom, Prenom, Email, Telephone, EmailConfirmed, LockoutEnd)
- AUDIT\_LOG (IdAuditLog, Action, Entite, EntiteId, DateAction, #IdUtilisateur)

## 5.5 Modèle Physique de Données (MPD)

Le modèle physique correspond à l'implémentation concrète dans SQL Server via Entity Framework Core (Code First). Il inclut :

- Définition des types SQL (INT, NVARCHAR, DATETIME2, DECIMAL, BIT, etc.)
- Contraintes d'intégrité (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE)
- Index automatiques sur les clés primaires et étrangères
- Gestion des migrations via Entity Framework Core
- Conventions de nommage : tables au pluriel (Actifs, Produits, etc.), clés primaires Id[Entité]

## 6. ARCHITECTURE TECHNIQUE

### 6.1 Architecture globale

L'application est développée en ASP.NET Core MVC avec une architecture en 4 couches distinctes, garantissant la séparation des responsabilités, la testabilité et l'évolutivité.

| Couche                     | Projet                   | Rôle                                                                                             |
|----------------------------|--------------------------|--------------------------------------------------------------------------------------------------|
| Domain                     | GestionInventaire.Domain | Entités métier, interfaces repositories, enums (StatutActif, TypeMouvement, StatutMaintenance)   |
| DAL — Data Access Layer    | GestionInventaire.DAL    | DbContext (Entity Framework Core), Repositories génériques et spécifiques, migrations SQL Server |
| BLL — Business Logic Layer | GestionInventaire.BLL    | Services métier, DTOs, règles de gestion, validations, orchestration des transactions            |
| Web — Présentation         | GestionInventaire.Web    | Controllers ASP.NET MVC, ViewModels, Vues Razor, AutoMapper profiles, Identity (auth + rôles)    |

### 6.2 Organisation en couches

#### Couche Domain (GestionInventaire.Domain)

Contient les entités métier (Actif, Produit, Categorie, Stock, MouvementStock, Service, Employe, Affectation, Maintenance, Utilisateur, AuditLog), les interfaces des repositories (IGenericRepository<T>, IActifRepository, IProduitRepository, etc.) et les enums (StatutActif, TypeMouvement, StatutMaintenance).

#### Couche DAL (GestionInventaire.DAL)

Contient AppDbContext (configuration Entity Framework Core), le GenericRepository<T> avec les méthodes GetAllAsync, GetByIdAsync, CreateAsync, Update, Delete, SaveAsync, et tous les repositories spécialisés. Contient également le AuditRepository avec IHttpContextAccessor pour tracer l'utilisateur connecté.

#### Couche BLL (GestionInventaire.BLL)

Contient tous les services métier (IActifService, IProduitService, IStockService, IEmployeService, IServiceService, IAffectationService, IMaintenanceService, IAuditService, IRapportService, IUtilisateurService, IDashboardService,

IHomeService). Les services ne contiennent aucune logique de présentation. Les controllers ne contiennent aucune logique métier.

### Couche Web (GestionInventaire.Web)

Contient les Controllers (zéro logique métier — délèguent aux services BLL), les ViewModels (DTOs de présentation), les Profiles AutoMapper (mapping DTO ↔ ViewModel), les Vues Razor (.cshtml), les fichiers CSS par module (wwwroot/css/), et la configuration Identity (Areas/Identity).

## 6.3 Flux de traitement

Le fonctionnement global suit ce flux :

- L'utilisateur interagit avec la Vue Razor (formulaire, bouton)
- Le Controller reçoit la requête HTTP, valide le ModelState, mappe le ViewModel en DTO via AutoMapper
- Le Controller appelle le Service BLL correspondant
- Le Service BLL applique les règles de gestion, orchestre les repositories, et appelle `AuditRepository.LogAsync`
- Les Repositories DAL exécutent les opérations SQL via Entity Framework Core
- Le Service retourne un DTO au Controller
- Le Controller mappe le DTO en ViewModel via AutoMapper et retourne la Vue

## 6.4 Technologies et outils

| Technologie                        | Usage                                                     |
|------------------------------------|-----------------------------------------------------------|
| ASP.NET Core MVC (.NET 8)          | Framework web, routing, controllers, vues Razor           |
| Entity Framework Core              | ORM, Code First, migrations, requêtes LINQ                |
| Microsoft SQL Server               | SGBD relationnel                                          |
| ASP.NET Identity                   | Authentification, gestion des rôles, gestion des comptes  |
| AutoMapper 16.x                    | Mapping automatique DTO ↔ ViewModel                       |
| IHttpContextAccessor               | Accès à l'utilisateur connecté depuis les couches DAL/BLL |
| Bootstrap Icons                    | Icônes de l'interface utilisateur                         |
| Google Fonts (DM Sans, Space Mono) | Typographie de l'interface                                |
| CSS Variables (thème sombre)       | Système de design cohérent sur tous les modules           |



## 7. CONTRAINTES ET HYPOTHÈSES

### 7.1 Contraintes techniques

- Architecture ASP.NET Core MVC 4 couches obligatoire : aucune logique métier dans les controllers, aucune logique de présentation dans les services
- Entity Framework Core : un DbContext par requête HTTP (Scoped) — les requêtes parallèles (Task.WhenAll) sur le même contexte sont interdites ; toutes les requêtes sont séquentielles
- Les transactions atomiques groupent les opérations liées (ex. création produit + stock + actifs) en un minimum de SaveAsync
- Les codes inventaire sont générés au format [Lettre][D6] supportant jusqu'à 999 999 actifs par produit

### 7.2 Contraintes de performance

- Le système doit gérer des volumes importants d'actifs (jusqu'à 10 000 actifs générés en une seule opération d'approvisionnement)
- Les rapports agrègent les données de tous les modules en mémoire après chargement séquentiel depuis la base

### 7.3 Contraintes de sécurité

- Authentification obligatoire pour accéder à toutes les pages (sauf login/invitation)
- Module Audit accessible uniquement aux Admins ([Authorize(Roles = "Admin")])
- Module Utilisateurs accessible uniquement aux Admins
- Protection CSRF sur tous les formulaires POST (AntiForgeryToken)
- Les mots de passe sont gérés par ASP.NET Identity (hachage BCrypt) — jamais stockés en clair

### 7.4 Hypothèses

- Les utilisateurs disposent d'un accès à un environnement informatique permettant l'utilisation de l'application web
- Les services/départements sont créés avant l'enregistrement des employés
- Les catégories sont créées avant l'enregistrement des produits
- Les produits sont créés avec une quantité initiale d'actifs lors de leur premier enregistrement
- L'email d'invitation est configuré dans l'environnement de production
- Les utilisateurs seront formés à l'utilisation du système

## 8. NORMALISATION DE LA BASE DE DONNÉES

Le modèle de données respecte les trois formes normales :

### 8.1 Première forme normale (1FN)

Toutes les entités contiennent des valeurs atomiques. Aucun attribut multivalué ou répété. Chaque entité dispose d'une clé primaire simple.

### 8.2 Deuxième forme normale (2FN)

Chaque attribut non-clé dépend entièrement de la clé primaire. La séparation Produit/Actif/Stock garantit l'absence de dépendances partielles : les attributs du produit (NomProduit, Description) sont dans la table Produit, les attributs de l'actif physique (CodeInventaire, Localisation, Statut) dans la table Actif, et les quantités dans la table Stock.

### 8.3 Troisième forme normale (3FN)

Les dépendances transitives ont été éliminées. Le NomService n'est pas stocké dans Employe (seulement IdService). Le NomCategorie n'est pas stocké dans Produit (seulement IdCategorie). Le NomProduit n'est pas stocké dans Actif (seulement IdProduit). Toutes les résolutions de noms se font par jointure ou dictionnaire en mémoire dans la couche BLL.